

Temario A

Un equipo de desarrollo de la Universidad está trabajando en un sistema que sustituya el actual sitio del entorno virtual de aprendizaje (EVA). Para un primer momento de la aplicación se quiere realizar todo lo necesario para el manejo de la información de los usuarios y el de evaluaciones.

Por cada usuario se conoce el nombre y facultad a la que pertenece. En el caso de que el usuario sea un estudiante debe tener además el grupo al que pertenece; si es un profesor se le debe agregar la asignatura que imparte. Por otro lado están los usuarios que son editores de cursos, que son profesores con la posibilidad de editar cursos y se les debe registrar si pueden o no dar permisos de edición a otros profesores.

Por cada evaluación que son realizadas por los estudiantes, se debe guardar el tipo de evaluación (tarea o cuestionario), la nota recibida y un comentario del profesor. El sistema debe permitir manejar las evaluaciones de forma general y de forma particular por cada estudiante.

Como la aplicación arroja un conjunto de información necesaria para otros sistemas, se debe dar la posibilidad tanto a los usuarios y las evaluaciones que puedan ser reportables, es decir, que se puedan utilizar para generar reportes.

Para promover la utilización de la aplicación se desea incluirle un ranking de usuarios, el cual estará determinado por las siguientes fórmulas:

Estudiantes: $\text{ranking} = cv * ce$	$cv \rightarrow$ Cantidad de visitas realizadas al sitio por el usuario
Profesores: $\text{ranking} = cv * cg$	$ce \rightarrow$ Cantidad de evaluaciones realizadas por el estudiante
Editores: $\text{ranking} = cv * cg * ca$	$cg \rightarrow$ Cantidad de grupos que atiende el profesor
	$ca \rightarrow$ Cantidad de cursos que ha creado el editor

Además se quiere saber la eficiencia del trabajo de los profesores, que es un valor entre 0-10 que se determina según con la cantidad y rapidez de los comentarios a las evaluaciones de los estudiantes; en el caso de que sea editor se le tiene en cuenta la cantidad de actividades que edita en los cursos.

Teniendo en cuenta la descripción anterior y el diagrama de clase que se modela a continuación, responda los siguientes incisos:

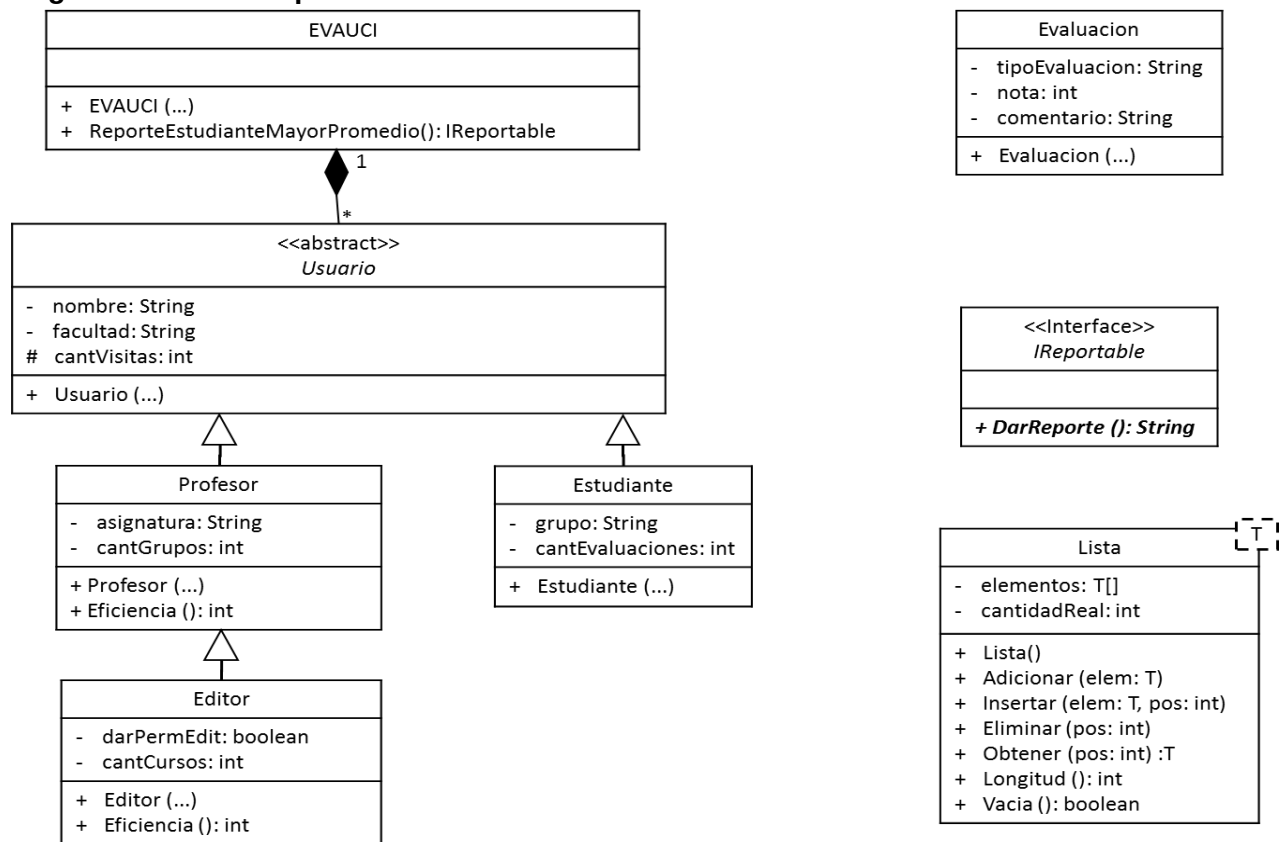
1. Complete el diagrama de clases dado, con las relaciones y métodos que faltan, para que se ajuste al problema descrito y se garantice que el método ***ReporteEstudianteMayorPromedio():IReportable*** sea funcional. El método retorna el reporte del estudiante con mayor promedio.
2. Implemente en las clases correspondientes los métodos polimórficos que no aparezcan en el diagrama en las clases.
3. Declare la clase controladora con sus atributos haciendo uso de la clase `Lista<T>` para la representación de los usuarios.
4. Implemente en la clase correspondiente:
 - a. El método ***CantTareasSinComentarios():int***, el cual retorna la cantidad de evaluaciones de tipo “Tarea” que aún no tienen comentarios.
 - b. El método ***UsuariosMasEficientes():Lista<Usuarios>***, el cual retorna una lista con los

usuarios que tienen una eficiencia mayor que 7.

5. Declare una clase genérica nombrada *ListaAlterna* y que herede de la clase *Lista<T>*. Agregue a esta el método *Extraer*, el cual retorna los elementos que se encuentran en un rango definido por las posiciones dadas por parámetro. Lance una excepción si no hay elementos en el rango dado.
6. Implemente en las clases correspondientes, los métodos necesarios para salvar en el fichero binario “*usuariosFICI.eva*” y separados por un cambio de línea: el nombre y la cantidad de visitas por cada usuario de la FICI. Realice el manejo de excepciones asociado al trabajo con fichero garantizando que siempre se cierre a pesar de que ocurra un error.

Nota: Asuma que todos los métodos de acceso a los atributos de las clases ya se encuentran implementados.

Diagrama de clases que modela la situación:

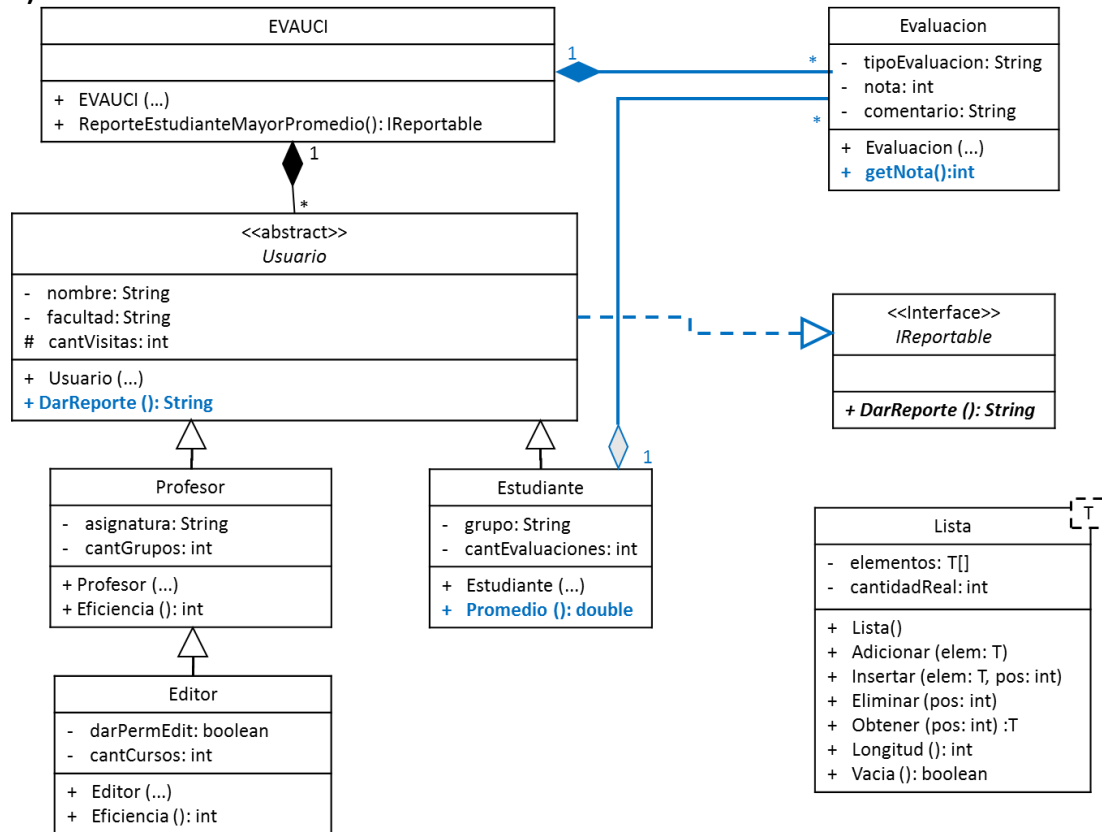


Clases para el trabajo con ficheros en Java:

File	FileReader	DataInputStream	BufferedReader	FileInputStream	ObjectInputStream
	FileWriter	DataOutputStream	BufferedWriter	FileOutputStream	ObjectOutputStream

Respuestas del Temario A

1)



2)

En la clase Usuario

```
public abstract double Ranking();
```

En la clase Estudiante

```
@Override
public double Ranking()
{
    return cantVisitas * cantEvaluaciones;
}
```

En la clase profesor

```
@Override
public double Ranking()
{
    return cantVisitas * cantGrupos;
}
```

En la clase editor

```
@Override
public double Ranking()
{
}
```

```

        return super.Ranking() * cantCursos;
    }

```

3)

```

public class EVAUCI
{
    Lista<Usuario> usuarios;
    Evaluacion[] evaluaciones;
    int cantEvaluaciones;
}

```

4)

```

a) public int CantTareasSinComentarios()
{
    int cant = 0;
    for (int i = 0; i < cantEvaluaciones; i++)
    {
        if(evaluaciones[i].getTipoEvaluacion().equals("Tarea") &&
            evaluaciones[i].getComentario().equals(""))
            cant++;
    }

    return cant;
}

```

b)

```

public Lista<Usuario> UsuariosMasEficientes()
{
    Lista<Usuario> eficientes = new Lista<Usuario>();
    for (int i = 0; i < usuarios.Longitud(); i++)
    {
        if(usuarios.Obtener(i) instanceof Profesor &&
            ((Profesor) usuarios.Obtener(i)).Eficiencia() > 7)
        {
            eficientes.Adicionar(usuarios.Obtener(i));
        }
    }

    return eficientes;
}

```

5)

```

public class ListaAlterna<T> extends Lista<T>
{

    public T[] Extraer(int pos1, int pos2) throws Exception
    {

```

```

T[] elems = (T[]) new Object[pos2 - pos1];

if (pos2 - pos1 <= 0)
{
    throw new Exception("No hay elementos en el rango especificado");
}

int cant = 0;
for (int i = pos1; i <= pos2; i++)
{
    elems[cant++] = Obtener(i);
}
return elems;
}
}

```

6)

```

public void GuardarFichero() throws Exception
{
    File file = new File("usuariosFICl.eva");
    FileWriter fileWriter = null;
    try
    {
        fileWriter = new FileWriter(file);
        for (int i = 0; i < usuarios.Longitud(); i++)
        {
            if(usuarios.Obtener(i).getFacultad().equals("FICIF"))
                fileWriter.write(usuarios.Obtener(i).getNombre() + " " + usuarios.Obtener(i).getCantVisitas());
        }
    } catch (IOException ex)
    {
        throw new Exception("Ocurrió un error en el momento de escribir en el fichero.");
    } finally
    {
        fileWriter.close();
    }
}

```

Temario B

Un equipo de desarrollo de la Universidad es designado para trabajar en un sistema informático que sea utilizado por el Museo de Bellas Artes para la gestión de sus obras de arte y locales de exposición. Por cada obra de arte se conoce el nombre, dimensiones (largo, ancho y altura) y el año de creación. En el caso de que la obra sea una pintura se registra el tipo de soporte (papel, lienzo, muro o madera) y el tipo de pigmento (si es orgánico, inorgánico o combinado); para el caso de que sea una escultura, el tipo de material (hierro, bronce, plomo, cera, yeso, plastilina, etc...) y el volumen que ocupa. Para el caso de los locales se desea registrar el tipo de obra que se exponen (pinturas, esculturas o mixtas), el nivel de iluminación que requiere (alto, medio o bajo) y todas las obras de arte que ahí se exponen.

Debido al transcurso del tiempo, tanto las obras de arte como los locales requieren ser restaurados cada cierto tiempo, por lo que la aplicación debe ser capaz de estimar cada que tiempo (en años) se deben restaurar.

Para el caso particular de las obras de arte para determinar el tiempo de restauración se debe tener en cuenta el factor de degradación de la obra que se calcula de la siguiente forma:

Pinturas: factor degradación = (año actual – año creación) * 1.5

Esculturas: factor degradación = (año actual – año creación) * 0.8

Por otro lado, también es de interés para el museo conocer el valor estimado en moneda nacional por lo que se valora la obra, para ello se tiene en cuenta el tipo de obra, los materiales que se utilizaron y su antigüedad.

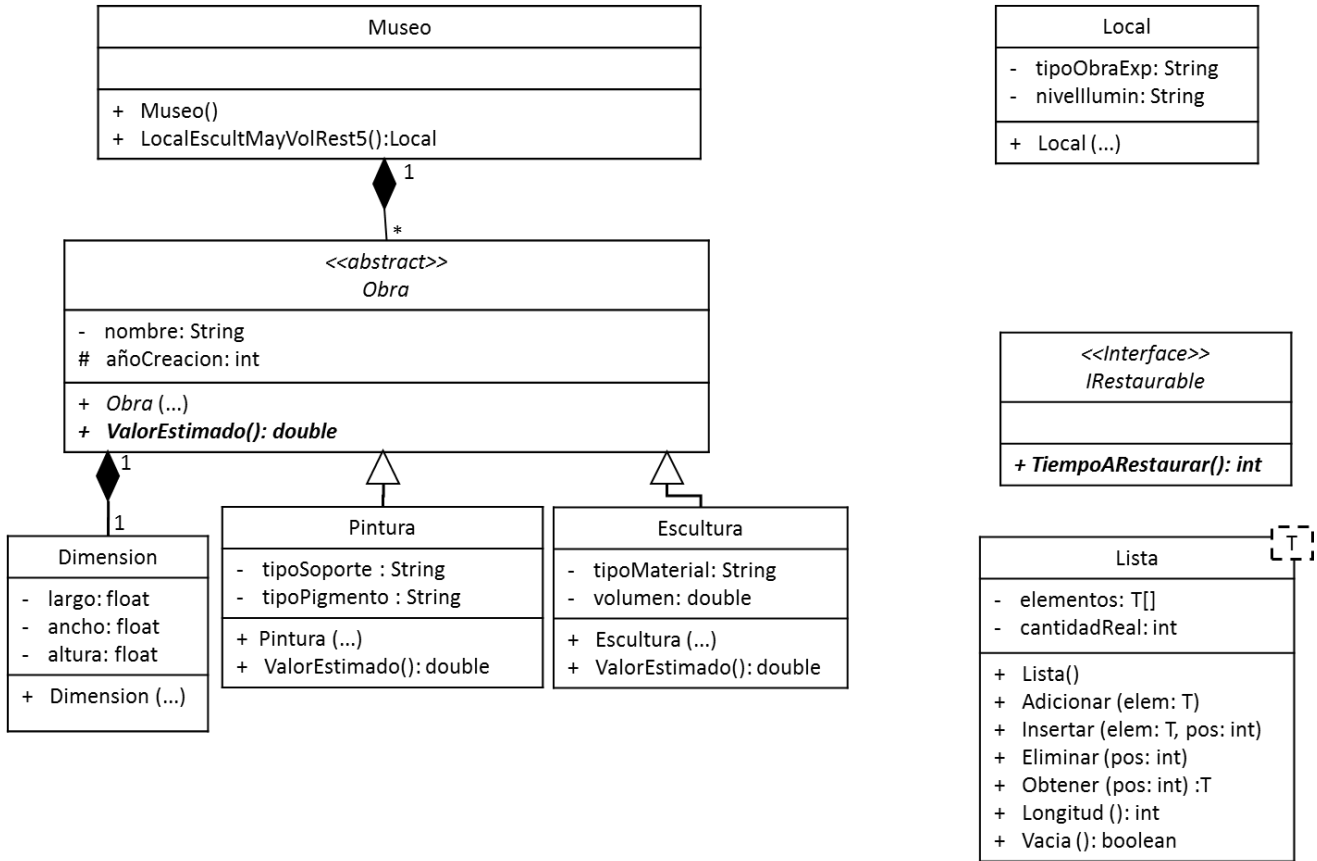
Teniendo en cuenta la descripción anterior y el diagrama de clase que se modela a continuación, responda los siguientes incisos:

1. Complete el diagrama de clases dado, con las relaciones y métodos que faltan, para que se ajuste al problema descrito y se garantice que el método **LocalEscultMayVolRest5():Local** sea funcional. El método retorna el local de solo esculturas que tiene la obra de arte de mayor volumen y con un tiempo de restauración de menos de 5 años.
2. Implemente en las clases correspondientes los métodos polimórficos que no aparezcan en el diagrama en las clases.
3. Declare la clase controladora con sus atributos haciendo uso de la clase Lista<T> para la representación de los obras.
4. Implemente en la clase correspondiente:
 - a) El método **ExistenLocalesEscultConIllum(nivel:String):boolean**, el cual retorna verdadero si existen locales de solo esculturas con el mismo nivel de iluminación dado por parámetro o falso en caso contrario.
 - b) El método **PinturasLienzoValoradas():Lista<Pintura>**, el cual retorna las pinturas de lienzo que tiene el museo que están valoradas por un precio superior a 1000 pesos.
5. Declare una clase genérica nombrada **ListaAlterna** y que herede de la clase Lista<T>. Agregue a esta el método **PrimerosN**, el cual retorna los elementos que se encuentran entre 0 y una posición dada por parámetro. Lance una excepción si la nueva lista no tiene elementos.
6. Implemente en las clases correspondientes, los métodos necesarios para salvar en el fichero binario “*obras90.mba*” y separados por un cambio de línea: el nombre y año de creación de todas las obras que son de antes de 1990. Realice el manejo de excepciones asociado al trabajo con

fichero garantizando que siempre se cierre a pesar de que ocurra un error.

***Nota:** Asuma que todos los métodos de acceso a los atributos de las clases ya se encuentran implementados.*

Diagrama de clases que modela la situación:

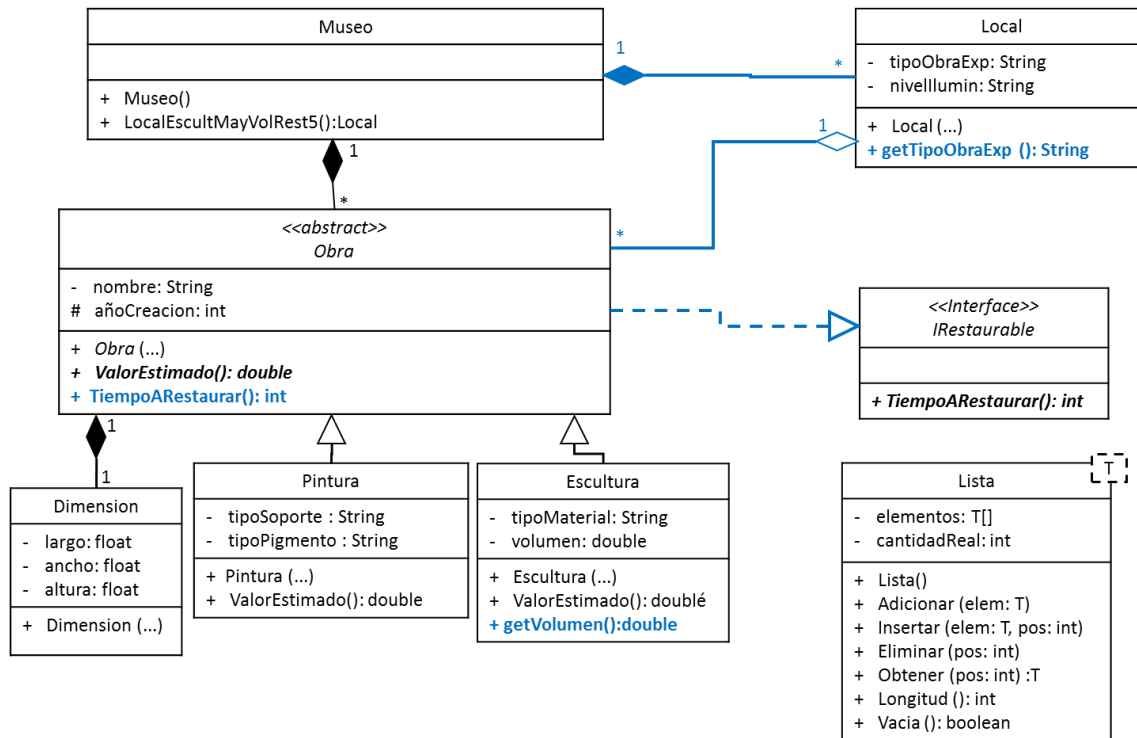


Clases para el trabajo con ficheros en Java

File	FileReader	DataInputStream	BufferedReader	FileInputStream	ObjectInputStream
	FileWriter	DataOutputStream	BufferedWriter	FileOutputStream	ObjectOutputStream

Respuestas del Temario B

1)



2)

En la clase Obra
 public abstract double FactorDegradacion();

En la clase Pintura
 @Override
 public double FactorDegradacion()
 {
 return (2016 - añoCreacion)*1.5;
 }

En la clase Escultura
 @Override
 public double FactorDegradacion()
 {
 return (2016 - añoCreacion)*0.8;
 }

3)

```

public class Museo
{
    Lista<Obra> obras;
    private Local[] locales;
    int cantLocales;
}
    
```

4)

a) public boolean ExistenLocalesEscultConIllum(String nivel)

```
{
    for (int i = 0; i < cantLocales; i++)
    {
        if (locales[i].getObraExp().equals("Escultura")
            && locales[i].getNivelIllumin().equals(nivel))
        {
            return true;
        }
    }
    return false;
}
```

b)

public Lista<Pintura> PinturasLienzoValoradas()

```
{
    Lista<Pintura> pinturas = new Lista<Pintura>();

    for (int i = 0; i < obras.Longitud(); i++)
    {
        if (obras.Obtener(i) instanceof Pintura
            && (obras.Obtener(i).ValorEstimado() > 1000))
            pinturas.Adicionar((Pintura) obras.Obtener(i));
    }
    return pinturas;
}
```

5)

public T[] PrimerosN(int pos) throws Exception

```
{
    T[] elems = (T[]) new Object[pos];

    int cant = 0;
    for (int i = 0; i <= pos; i++)
    {
        elems[cant++] = Obtener(i);
    }

    if (cant == 0)
    {
        throw new Exception("La lista nueva no tiene elementos");
    }

    return elems;
}
```

6)

```
public void GuardarFichero() throws Exception
{
    File file = new File("obras90.mba");
    FileWriter fileWriter = null;
    try
    {
        fileWriter = new FileWriter(file);
        for (int i = 0; i < obras.Longitud(); i++)
        {
            if (obras.Obtener(i).getAñoCreacion() < 1990)
            {
                fileWriter.write(obras.Obtener(i).getNombre() + " " + obras.Obtener(i).getAñoCreacion());
            }
        }
    } catch (IOException ex)
    {
        throw new Exception("Ocurrió un error en el momento de escribir en el fichero.");
    } finally
    {
        fileWriter.close();
    }
}
```